#Coding

Copilot Diagnostics Toolset for .NET in Visual Studio

#4<DEV>

Mirco Vanini



MVP Microsoft® Most Valuable Professional



AGIC





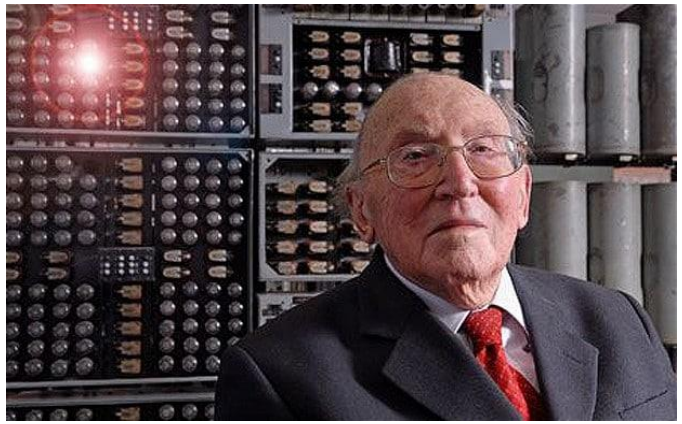
BizzyNow™

make yourself better

READY2USE

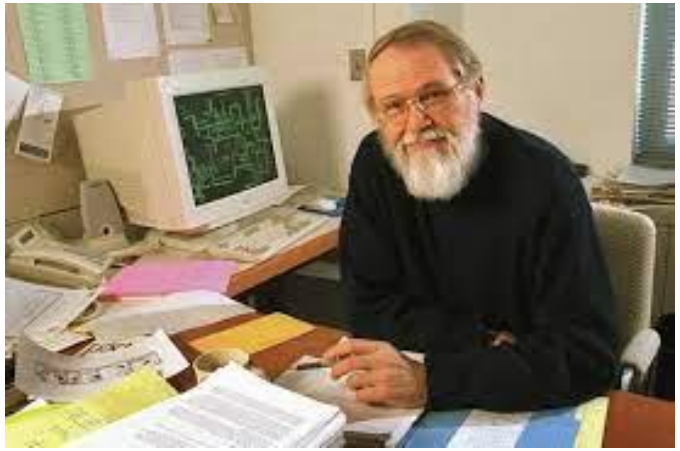
"As soon as we started programming, we found to our surprise that it wasn't as easy to get programs right as we had thought. Debugging had to be discovered. I can remember the exact instant when I realized that ***a large part of my life*** *from then on* ***was going to be spent in finding mistakes*** *in my own programs."*

Sir Maurice Wilkes



"Debugging is ***twice as hard*** *as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."*

Brian Kernighan



An integrated set of tools powered by Copilot to analyze, diagnose, and optimize .NET applications directly within Visual Studio.

Explain code

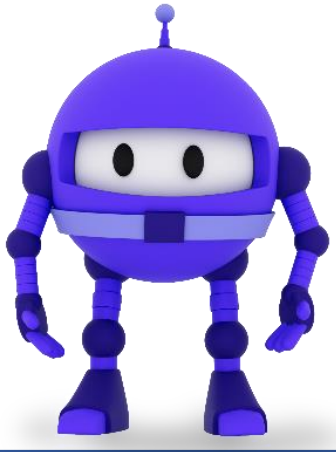
Exception

Breakpoints and tracepoints

Variables

LINQ query analysis

Quick Actions



Fast, Lightweight Edits

- Quick explanations
- JSON/YAML transformations
- Small refactors
- Regex generation
- Short Q&A tasks

Lightweight models (e.g., MAI-Code-1.1-Flash, Claude Haiku 4.5, Raptor mini)

These models give near-instant responses and keep you “in flow.”

Everyday Development Tasks

- Writing new functions
- Improving readability
- Generating tests
- Creating documentation

General-purpose coding models (e.g., GPT-5-mini, GPT-5.3-Codex, GPT-5.6 Luna, Claude Sonnet 5)

These models offer the best balance between speed and quality.

Complex Debugging & Deep Reasoning

- Analyzing unfamiliar code
- Debugging tricky production issues
- Architecture decisions
- Multi-step reasoning
- Performance analysis

Deep reasoning models (e.g., GPT-5.5, GPT-5.6 Sol, Claude Sonnet 5, Gemini 3.1 Pro, Claude Opus 4.7 / 4.8 / 5)

These models handle large context, produce structured reasoning, and give the most reliable insights for complex engineering tasks.

GitHub Copilot Models - Developer Friendly Comparison

Model Category	Example Models (Premium request Multiplier for paid plans)	What they're best at	When to Use Them
Fast Lightweight Models	MAI-Code-1.1-Flash, Claude Haiku 4.5, Gemini 3.5 Flash, Raptor mini	Low latency, quick responses	Small edits, Q&A, simple code tasks
General-Purpose Coding Models	GPT-5-mini, GPT-5.6 Terra GPT-5-Codex, Claude Sonnet 5,	Reliable day-to-day development	Writing functions, small tests, documentation
Deep Reasoning Models	GPT-5, GPT-5.6 Sol, Claude Sonnet 5, Claude Opus 4.7 / 4.8, Claude Opus 5	Complex reasoning and debugging	Architecture work, deep bug diagnosis
Agentic / Multi-step Models	GPT-5.3-Codex, GPT-5.4 mini, GPT-5.6 Terra, Claude Sonnet 5, Grok 4.5 / 4.6, Kimi K3	Planning + execution workflows	Repo-wide changes, feature scaffolding

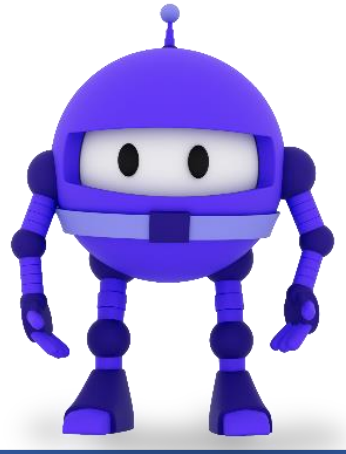
GitHub Copilot's ***Auto*** model selection automatically picks the best available model for your prompts, balancing performance and cost. It chooses from models like GPT-5.5 and Claude Haiku 4.5 based on availability, your subscription, and restrictions, while avoiding blocked or premium models. Paid users also get a 10% discount on premium requests in Copilot Chat.

AI model comparison

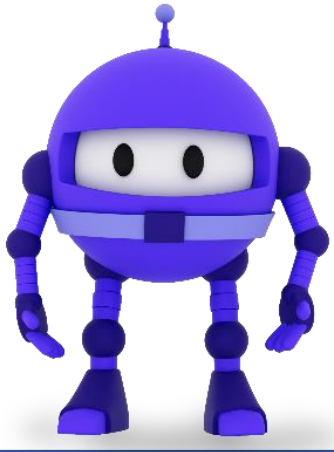
About Copilot auto model selection

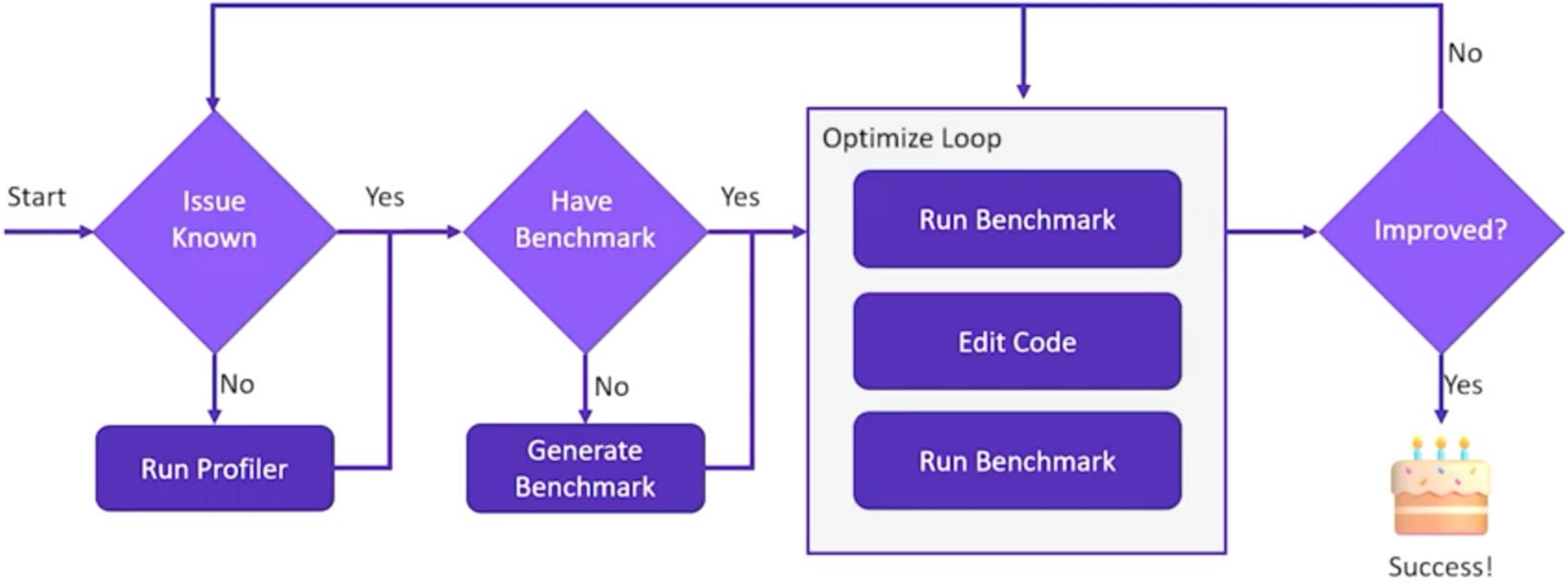
Choosing the Right Model in GitHub Copilot: A Practical Guide for Developers

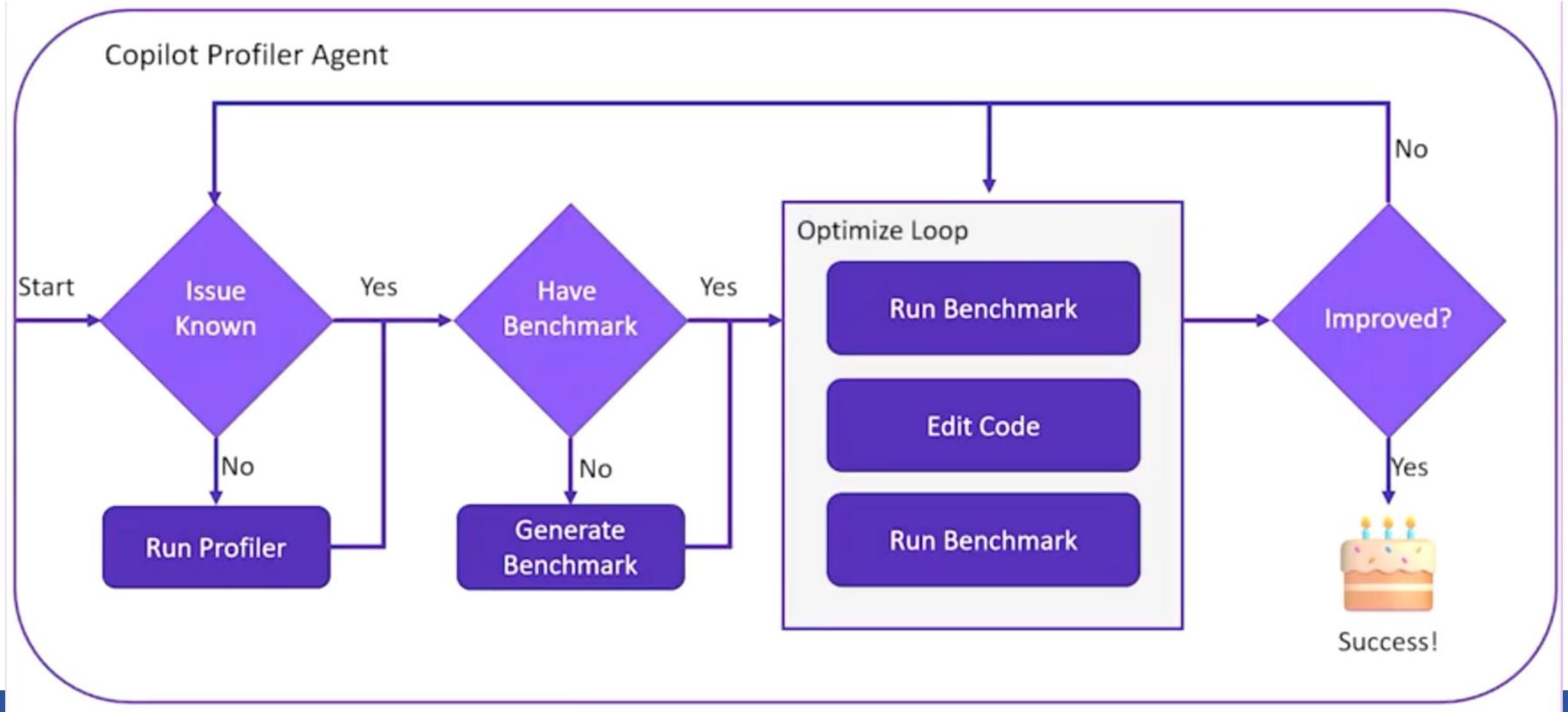
Multithreaded debugging



Profiling



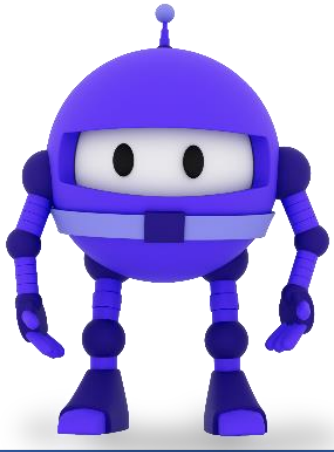

Start
Issue Known
Yes
No
Run Profiler
Have Benchmark
Yes
No
Generate Benchmark
Optimize Loop
Run Benchmark
Edit Code
Run Benchmark
Improved?
No
Yes
Success!


Copilot Profiler Agent
Start
Issue Known
Yes
No
Run Profiler
Have Benchmark
Yes
No
Generate Benchmark
Optimize Loop
Run Benchmark
Edit Code
Run Benchmark
Improved?
No
Yes
Success!



Profile agent crash with Unhandled exception. System.IO.IOException: The pipe is being closed on Win11 Arm

Unit testing



@Test (Preview) C# #DemoClass.cs

> 1 reference

Test (Preview)

> 1 reference

Test generation can be a long-running process. Depending on the scope of your target, this may take a while.

- ✓ Analysis and planning completed.
- ✓ Using existing test project 'DemoClassLibrary.UnitTests' for source project 'DemoClassLibrary'
- ✓ Collected initial coverage for test project 'DemoClassLibrary.UnitTests'
- ✓ Created test file 'DemoClassTests.cs'
- ✓ Fixed failing tests for project 'DemoClassLibrary.UnitTests'
- ✓ Measuring coverage improvements

Test (Preview)

> 0 references

Summary of Test Generation

Status: Completed Successfully

- **Total Unit Tests Generated:** **79** tests
- **Files Created/Modified:** **1** files
- **Test Projects Touched:** **1** projects
  - Projects: DemoClassLibrary.UnitTests.csproj

Test Coverage Impact

- **Initial Coverage:** N/A
- **Final Coverage:** **100.0%**
- **Coverage Change:** N/A

Quality Indicators

- **Pass/Fail Summary:** **73 passed**, **3 skipped** (production bug suspected)

🔍 Insights & Recommendations

- The 3 skipped tests are tied to a production issue: serializer options do not allow named floating-point literals (NaN/Infinity). Update SerializerOptions to include NumberHandling = JsonNumberHandling.AllowNamedFloatingPointLiterals.
- After applying the fix, re-run the test suite to convert skipped tests into passing ones and confirm stability.
- Consider adding a regression test specifically for special floating-point values (NaN/Infinity/-Infinity) to prevent recurrence.

⚠ Potential Bugs Detected

- Identified issue: SerializerOptions in src\DemoClassLibrary\DemoClass.cs is missing JsonNumberHandling.AllowNamedFloatingPointLiterals. This prevents serializing special double values and caused test skips.
- Suggested fix:
  - Add NumberHandling = JsonNumberHandling.AllowNamedFloatingPointLiterals to SerializeHelper.SerializerOptions.
  - Example change:
    - NumberHandling = JsonNumberHandling.AllowNamedFloatingPointLiterals

🔍 **Rapid** identification of performance and runtime issues

⚙️ **Intelligent** suggestions for debugging and code optimization

📊 **Clear** visualizations of metrics and execution flows

🚀 **Proactive** support to enhance quality, stability, and developer productivity



Q&A Time!



Mirco Vanini

Microsoft MVP Developer Technologies

Consultant focused on industrial and embedded solutions using .NET and other native SDKs with over 37 years of experience, XeDotNet community co-founder, speaker and Microsoft MVP since 2012

Microsoft® MVP Most Valuable Professional

sessionize MOST ACTIVE SPEAKER 2023

sessionize MOST ACTIVE SPEAKER 2024

sessionize MOST ACTIVE SPEAKER 2025

@MircoVanini
www.proxsoft.it
https://www.linkedin.com/in/proxsoft
https://sessionize.com/mirco-vanini



.NET
your platform for building anything